第 3 章練習詳解：行程

OS03-E01　行程狀態轉移

1. P1 發出 I/O 而由 running 進入 waiting；排程器選中 P2，P2 由 ready 進入 running。磁碟完成時，P1 由 waiting 轉為 ready。
2. P1 阻塞時可切換至 P2；完成事件後，若排程策略允許搶占，也可能切回 P1。非搶占式策略下，P2 可先繼續執行。
3. 不一定。喚醒只代表 P1 已具備執行資格；仍須經排程器選取及 dispatcher 恢復上下文。單 CPU 模型同一時刻最多一個行程 running。

OS03-E02　Zombie 與 orphan

1. 子行程已結束、父行程尚未收取其結束狀態時，子行程會留下 zombie 狀態。單獨呼叫 `fork` 或 `exec` 並不會產生 zombie。
2. 父行程先結束，子行程仍存活時，該子行程成為 orphan；系統通常依其政策重新指定收養者。
3. 父行程須處理子行程結束並呼叫適當的 `wait` 類操作。多子行程的服務可使用訊號或集中式 reaper 安排回收，避免主服務無限期阻塞。

OS03-E03　有界通道與結束協定

1. J1、J2 已占滿兩格。阻塞 send J3 會等到 receive 釋出一格；非阻塞 send 則回傳 would-block，且不改變佇列。成功入列不等於工作效果已完成。
2. 不可以。本題明定操作以完整訊息為原子單位。Byte stream 或實際 pipe 的短寫規則需另外依其介面、長度與阻塞模式判定。
3. 停止接收新工作，依 FIFO 送 STOP；consumer 完成之前的工作後取出 STOP 並確認，管理者再清理。兩個 consumer 若只有一個 STOP，另一個可能持續等待；可各送一個，或設計關閉狀態及廣播。

OS03-E04　上下文與物件身分

1. 保存 P100 的 PC、暫存器與堆疊指標等架構所需狀態，再載入 P101 的 0x120、3、0xA000 及對應的記憶體管理狀態。P100 再被選中時，恢復 P100 自己的上下文；保存 PC 的精確位置取決於架構與事件。
2. 不能。唯讀程式碼頁可共用，私有 heap、stack 或全域變數仍可各自獨立；相同虛擬位址可映射不同實體頁；相同 fd 編號也可引用不同物件。
3. 不必然。系統呼叫進出核心可只改變處理器模式；同一行程內切換執行緒也可以更換暫存器及堆疊，同時共用使用者位址空間。

OS03-E05　遺失喚醒

1. 等待者看到 `done=false` 後暫停；完成方設 `done=true`，但此時佇列仍空；等待者接著才登記並睡眠。若沒有下一次完成事件，就可能永久等待。
2. 共同鎖保護 `done` 和登記；等待者以 `while` 檢查條件，再以原子 `park_and_release(L)` 交接等待與釋鎖。完成方持同一鎖設定 `done=true` 並喚醒。完成先取得鎖時，等待者看到條件成立而不睡；等待者先登記時，完成方在原子交接後可找到它並喚醒。僅在睡前重查 WAITING，仍會留下重查後、睡眠前的競賽。
3. 喚醒只使等待者具備執行資格；仍須由 scheduler 選取並 dispatch。

OS03-E06　Fork 樹與共享 offset

1. 第一次 P0 建立 P1；第二次 P0 建立 P2、P1 建立 P3，共四個。若每個現有行程都到達下一次成功的 fork，則 N(0)=1、N(k+1)=2N(k)，所以 N(k)=2^k。若子行程提前離開、分支限制呼叫者或 fork 失敗，前提不成立。
2. 父行程讀取前 offset＝0、讀取後為 100；子行程讀取前為 100、讀取後為 150。這是因為兩個描述符表項引用同一開啟檔案紀錄 O1；另行 open 可得到有獨立 offset 的紀錄。
3. 成功 exec 取代同一行程的映像，不增加行程數。子行程結束且狀態尚未被收取時是 zombie；父行程先結束、子行程仍存活則涉及 orphan。wait 必須先辨認正常結束，再從狀態解出 7，原始 status word 不一定直接等於 7。

OS03-E07　SPSC 環形佇列與發布

1. 以 `(head,tail) FIFO` 記錄：`(0,0) [] → (0,1) [A] → (0,2) [A,B]`；第一次嘗試 C 時已滿，狀態不變。接著 get A 得 `(1,2) [B]`，put C 得 `(1,0) [B,C]`，再依序 get B、get C，得到 `(2,0) [C] → (0,0) []`。
2. Producer 先寫 slot，再 release 發布 tail；consumer 透過對應 acquire 觀察發布後讀取資料。Consumer 先讀完 slot，再 release 發布 head；producer 透過對應 acquire 觀察釋放後才重用。兩個方向分別保護資料發布與槽位重用。
3. 不能。普通共享整數的衝突存取可構成 C/C++ data race；兩個 producer 也會競爭同一 tail 和槽位。跨行程原子操作另需平台支援。有限追蹤可核對索引，但不能取代一般同步論證。

OS03-E08　串流 framing 與共享物件

1. 第一批只有半個 header，不輸出；第二批補齊長度 3，但只有 `AB`，仍不輸出；第三批先補 `C`，輸出 `ABC`，再解析長度 2 並輸出 `DE`。讀取批次不是紀錄邊界。
2. 第四個 byte 後只有 header 和 `AB`，距離長度 3 還差一 byte，因此 EOF 代表截斷。長度 9 超過上限 8，應在配置或接收該 payload 前拒絕。
3. offset 16 是 0x10，兩個位址分別為 0x100010、0x700010。關閉 descriptor 不自動解除映射；unlink 移除名稱，既有映射不因此立即失效。初始化完成的發布仍需同步。
4. 不等同。`mq_unlink` 移除名稱，既有引用可持續到最後關閉；System V `IPC_RMID` 移除佇列，阻塞操作會被喚醒並收到 `EIDRM`。排空及停止必須另訂協定。

OS03-E09　RPC 重試與編碼

1. 77＝0x4D、1＝0x0001、25＝0x19；big-endian 編碼為 `00 00 00 4D | 00 01 | 00 00 00 19`，共 4＋2＋4＝10 bytes。
2. 請求未到達時餘額仍為 100；也可能 server 已提交扣款、餘額為 75，但回覆遺失。Client 兩種情況都看到 timeout。若使用新 ID 盲目重試，後者可能再扣成 50；若完全不重試，前者的工作可能遺漏。
3. 以經驗證的 client 身分及 request ID 查詢去重結果；把扣款與結果紀錄放在同一可恢復的原子交易。重送同一 ID 時返回既有結果 75。並行重複請求還須使用交易或唯一性約束，避免兩者各自查無紀錄後都扣款。
4. RAM 紀錄可能在重啟後消失；效果與紀錄分開寫入時，兩次寫入之間故障可導致重複扣款；紀錄刪除後若舊 ID 仍可重試，也可能再次執行。需共同持久提交，並約定 ID、epoch 與重試／保留期限。At-most-once 仍允許零次執行。

OS03-E10　七態模型的事件順序

依序為 running → blocked → blocked-suspended → ready-suspended → ready → running。完成事件只消除等待條件，不能直接讓仍在外部儲存的 A 執行。若先換入，則 blocked-suspended → blocked → ready；要變成 running 還需要 dispatch。

OS03-E11　睡前重查的競賽

等待者讀到 WAITING 後暫停；完成方令條件成立並喚醒；等待者恢復後依先前判斷睡下，沒有第二次喚醒便可能永久等待。釋放鎖與提交等待，必須相對完成方的條件更新及喚醒構成原子交接，或者以等價協定記住提前發生的喚醒。返回後須在鎖下以 while 重查條件。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 3. read(2); pipe(7); msgctl(2); mq_unlink(3).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study